

BE ↔ LSU 接口规范 v4

be_lsu_*	由 BE 驱动、LSU 采样	前缀 = 该线自身方向，与 RTL 端口 input/output 一致
lsu_be_*	由 LSU 驱动、BE 采样	

1. 信号表

1.1 时钟与复位

方向	信号	位宽	说明
—	clk	1	全局时钟
BE → LSU	rst_n	1	低有效。复位期间 LSU 清空全部在飞状态，lsu_be_issue_ready 驱 0

1.2 Issue 通道

方向	信号	位宽	说明
BE → LSU	be_lsu_issue_valid	1	访存指令交付，每拍至多一条
LSU → BE	lsu_be_issue_ready	1	本拍可接收；已包含 LSU 全部内部资源限制，BE 不再看第二个 ready
BE → LSU	be_lsu_issue_pld	—	仅在握手成功时采样，见 §2

1.3 正常完成通道

方向	信号	位宽	说明
LSU → BE	lsu_be_done_valid	1	该 tag 正常完成，BE 据此标 done
BE → LSU	be_lsu_entry_ready	1	与 §1.4 共用。除复位与 global_flush 同拍外恒为 1，不产生背压
LSU → BE	lsu_be_done_pld	—	与 valid 同拍有效，见 §3

1.4 异常通道

方向	信号	位宽	说明
LSU → BE	lsu_be_exception_valid	1	该 tag 访存故障，BE 据此拉高 exception
BE → LSU	be_lsu_entry_ready	1	同 §1.3，同一根线
LSU → BE	lsu_be_exception_pld	—	与 valid 同拍有效，见 §4

两条完成通道的约束：

每个 self_tag 恰好一个终态: lsu_be_done_valid 或 lsu_be_exception_valid, 二者互斥
两条 valid 不得同拍拉高——LSU 每拍至多交回一笔完成
store / FENCE / FENCE.I 无 rd, 正常完成同样走 § 1.3, lsu_be_done_data = 0
store_side (store / AMO / SC) 的 lsu_be_done_valid 在写内存完成之后才发

1.5 Bypass 通道

方向	信号	位宽	说明
LSU → BE	lsu_be_bypass_valid	1	与 lsu_be_done_valid 同拍发出; 无 ready, 单拍广播, 不可重发
LSU → BE	lsu_be_bypass_pld	—	tag / data 取值与 lsu_be_done_pld 相同

lsu_be_bypass_valid = lsu_be_done_valid && read_side
store / FENCE / FENCE.I 的正常完成不发 bypass——无 rd, 无人等它的 tag
异常不发 bypass
无 ready 且不可重发, 因此只在 done 握手成功的那一拍发出

1.6 Store wakeup 通道

方向	信号	位宽	说明
BE → LSU	be_lsu_store_wakeup_valid	1	一拍脉冲, 普通 store 的非推测授权, 见 §5

1.7 Flush

方向	信号	位宽	说明
BE → LSU	global_flush	1	一拍脉冲, 见 §6

2. be_lsu_issue_pld

字段	位宽	说明
self_tag	4	关联 issue / wakeup / 完成的唯一句柄，取值即 ROB/SCB 槽号
req_property	7	互斥 one-hot 分类，见 §2.1
exe_subop	24	指令身份，见 §2.2
mem_func3	3	RISC-V load/store funct3; [1:0] 给访问字节数 $1 \ll \text{func3}[1:0]$
rd_is_fp	1	结果写 FPR; 仅 read_side 有效
rs1_data	64	基址 / 原子地址
rs2_data	64	store 数据、SC 写数据或 AMO 操作数; 仅 store_side 有效
imm_valid	1	立即数是否有效; 原子指令与 FENCE 为 0
imm_data	64 (有符号)	最终字节偏移，已符号扩展，压缩指令的缩放由 BE 完成
is_store	1	兼容位，恒等于 req_property.is_store
st_br_resolve	1	alloc 那一拍冻结的授权快照，见 §5; 非普通 store 恒为 0

地址由 LSU 内部 AGU 计算：

```
vaddr = imm_valid ? (rs1_data + imm_data) : rs1_data      // 64 位模  $2^{64}$  回绕
```

对齐判定、符号扩展、NaN-boxing 由 ISA Model 完成，LSU 不预判、不拆分访问。

2.1 req_property

```
req_property = {is_load, is_store, is_amo, is_lr, is_sc, is_fence, is_fence_i}

read_side  = is_load | is_lr | is_amo | is_sc      有结果回写
store_side = is_store | is_amo | is_sc             写内存（LR 不算）
misc       = is_fence | is_fence_i                无回写、无写内存
```

2.2 exe_subop

```
exe_subop[23:0] = {format[1:0], opcode_or_op[6:0], funct3[2:0], high_fixed[11:0]}

format = 2'b01   普通 32-bit 指令
format = 2'b10   RVC 16-bit 指令
```

固定 funct7 / funct5 / funct12 投影到 high_fixed ; 可变立即数、寄存器号、rm、aq/r1 不进入 key。

2.3 exe_subop 冻结表

W = 访问字节数，R = 有 read-side 结果，S = 写内存。

loads (15)

exe_subop	property	mem_funct3	W	R	FP result
SUBOP_LB	load	000	1	Y	N
SUBOP_LH	load	001	2	Y	N
SUBOP_LW	load	010	4	Y	N
SUBOP_LD	load	011	8	Y	N
SUBOP_LBU	load	100	1	Y	N
SUBOP_LHU	load	101	2	Y	N
SUBOP_LWU	load	110	4	Y	N
SUBOP_FLW	load	010	4	Y	Y
SUBOP_FLD	load	011	8	Y	Y
SUBOP_C_LW	load	010	4	Y	N
SUBOP_C_LD	load	011	8	Y	N
SUBOP_C_FLD	load	011	8	Y	Y
SUBOP_C_LWSP	load	010	4	Y	N
SUBOP_C_LDSP	load	011	8	Y	N
SUBOP_C_FLDSP	load	011	8	Y	Y

全部 memop = LOAD, S 恒为 N。FP result = Y 即 rd_is_fp = 1。

stores (12)

exe_subop	property	mem_funct3	W	R	S
SUBOP_SB	store	000	1	N	Y
SUBOP_SH	store	001	2	N	Y
SUBOP_SW	store	010	4	N	Y
SUBOP_SD	store	011	8	N	Y
SUBOP_FSW	store	010	4	N	Y
SUBOP_FSD	store	011	8	N	Y
SUBOP_C_SW	store	010	4	N	Y
SUBOP_C_SD	store	011	8	N	Y
SUBOP_C_FSD	store	011	8	N	Y
SUBOP_C_SWSP	store	010	4	N	Y
SUBOP_C_SDSP	store	011	8	N	Y
SUBOP_C_FSDSP	store	011	8	N	Y

全部 memop = STORE。仅这 12 条 req_property.is_store = 1，也只有它们走 \$5 的 wakeup 授权。

LR / SC / AMO (22)

exe_subop	property	W	R	S	memop
SUBOP_LR_W, SUBOP_LR_D	lr	4, 8	Y	N	LR
SUBOP_SC_W, SUBOP_SC_D	sc	4, 8	Y	Y	SC
SUBOP_AMOSWAP_W, SUBOP_AMOSWAP_D	amo	4, 8	Y	Y	AMOSWAP
SUBOP_AMOADD_W, SUBOP_AMOADD_D	amo	4, 8	Y	Y	AMOADD
SUBOP_AMOXOR_W, SUBOP_AMOXOR_D	amo	4, 8	Y	Y	AMOXOR
SUBOP_AMOAND_W, SUBOP_AMOAND_D	amo	4, 8	Y	Y	AMOAND
SUBOP_AMOOR_W, SUBOP_AMOOR_D	amo	4, 8	Y	Y	AMOOR
SUBOP_AMOMIN_W, SUBOP_AMOMIN_D	amo	4, 8	Y	Y	AMOMIN
SUBOP_AMOMAX_W, SUBOP_AMOMAX_D	amo	4, 8	Y	Y	AMOMAX
SUBOP_AMOMINU_W, SUBOP_AMOMINU_D	amo	4, 8	Y	Y	AMOMINU
SUBOP_AMOMAXU_W, SUBOP_AMOMAXU_D	amo	4, 8	Y	Y	AMOMAXU

必须自然对齐（.W 4 字节、.D 8 字节）；未对齐由模型判定并报 cause 4 / 6，LSU 不预判。

fence (2)

exe_subop	property	memop	R	退休后动作
SUBOP_FENCE	fence	FENCE	N	正常提交
SUBOP_FENCEI	fence_i	FENCE.I	N	SCB 提交后由 flush_model 重定向至 PC+4 并作废 I-cache

汇总

```
read_side = 1    load 15 + LR 2 + SC 2 + AMO 18 = 37
store_side = 1    store 12 + SC 2 + AMO 18      = 32
misc      = 1     FENCE 1 + FENCE.I 1          = 2
```

AMO 的 .w 结果取 ISA Model 的 RV64 值；FP load 由模型返回已 NaN-boxing 的 64 位数据。**Buffer 不做符号扩展、不做 NaN-boxing。**

3. lsu_be_done_pld

字段	位宽	说明
lsu_be_done_tag	4	目标 self_tag
lsu_be_done_data	64	读侧结果，已完成整数符号 / 零扩展或 FP NaN-boxing；无 rd 的正常完成为 0

4. lsu_be_exception_pld

字段	位宽	说明
lsu_be_exception_tag	4	出错指令的 self_tag
lsu_be_exception_cause	5	RISC-V 标准同步异常号，不带中断标志位
lsu_be_exception_tval	64	出错虚地址

cause 由模型返回：

```
不在标准同步异常号内，或地址翻译查询失败时按写侧回退：
store_side = 1    → 7    (Store/AMO access fault)
其余          → 5    (Load access fault)
tval 回退为虚地址

预期取值
4 Load address misaligned      6 Store/AMO address misaligned
5 Load access fault            7 Store/AMO access fault
13 Load page fault             15 Store/AMO page fault
```

6. Flush 与复位的清除范围

global_flush	LSU	清空全部在飞条目、提前 wakeup 暂存与待发终态
	BE	拉低 be_lsu_issue_valid、be_lsu_store_wakeup_valid 并把 be_lsu_entry_ready 拉低一拍
rst_n = 0		同上，另外 lsu_be_issue_ready 驱 0

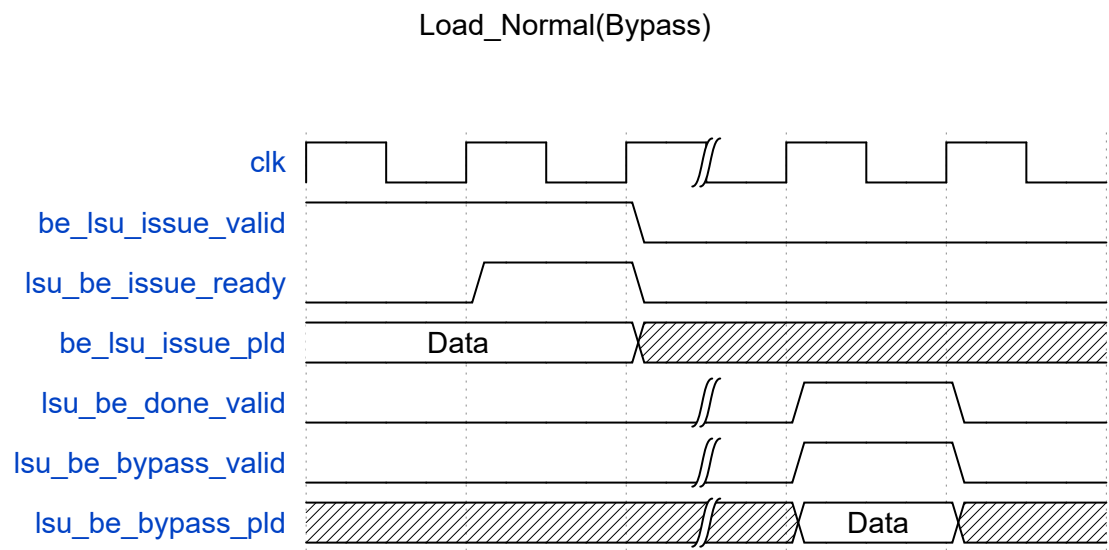
6.1 flush 同拍的事务

be_lsu_issue	不接收——该笔视为未发生，BE 不重发
be_lsu_store_wakeup	不接收
lsu_be_done / lsu_be_exception	不接收；LSU 直接丢弃，不得在下一拍重驱
lsu_be_bypass	同拍不发出

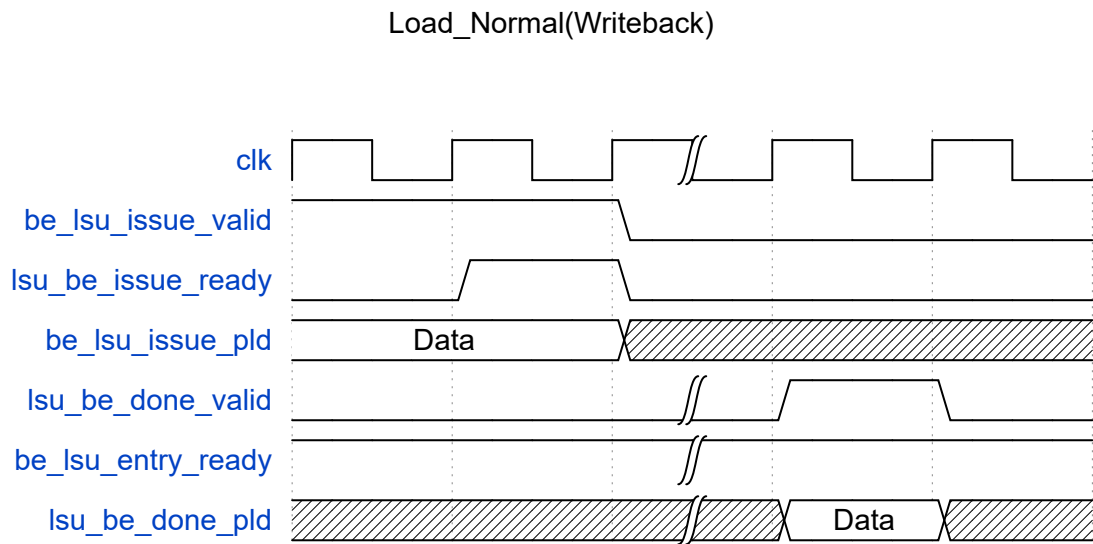
被丢弃的 self_tag 立即可复用，无恢复拍数。

7. 场景时序

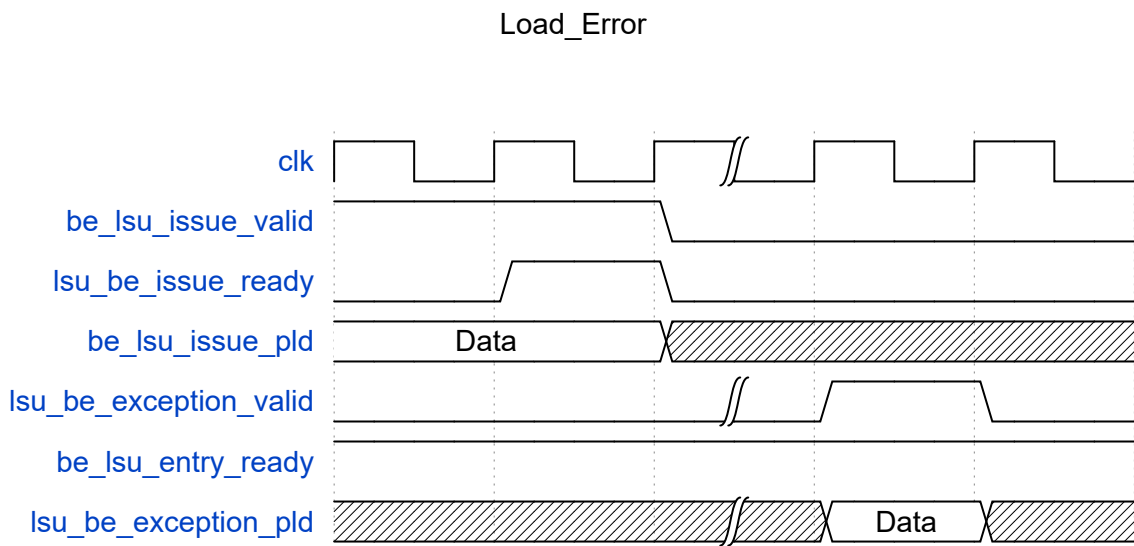
7.1 load — 无异常 (Write_Back)



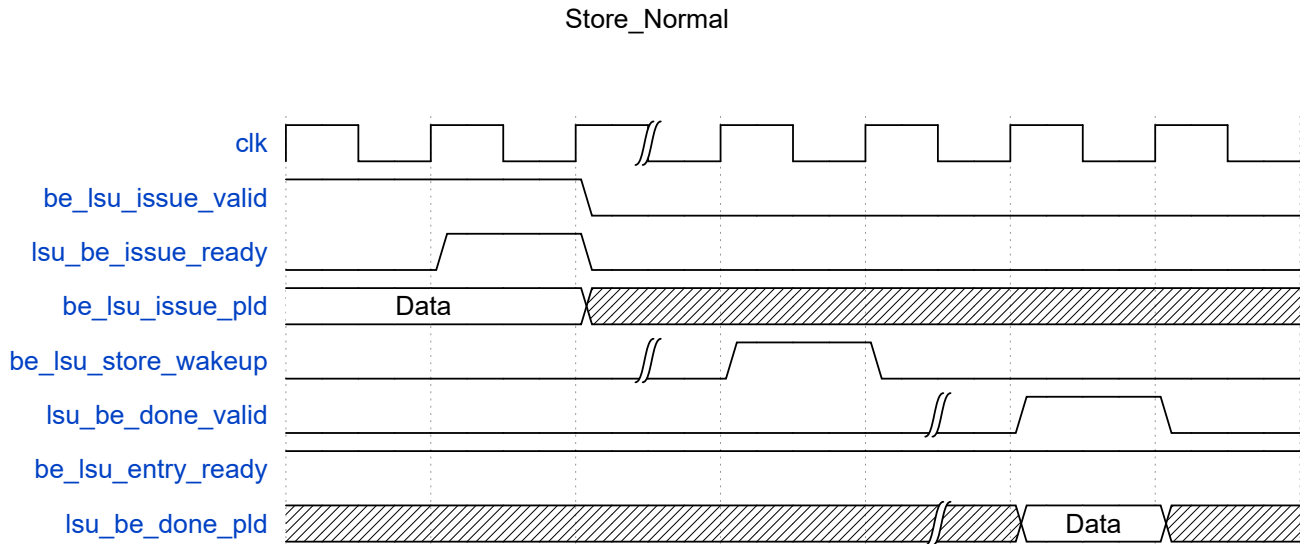
7.2 load — 无异常 (WriteBack)



7.3 load — 异常



7.4 Store — 无异常



7.5 Store — 异常

Store_Error

